

Modelado de requerimientos

Prototipación de requerimientos

Un prototipo es un modelo o maqueta que muestra las interfaces entre el sistema y el usuario. El prototipo simulará los aspectos más visuales del sistema, es decir, entradas y salidas, los componentes o elementos gráficos participantes, y los conceptos a tratar. Los prototipos simulan la interactividad del sistema con los usuarios, y dichas interacciones deben desarrollarse con cuidado, buscando la mayor similitud al sistema final.

Algunas pautas a tener en cuenta para su desarrollo:

- Se debe terminar a qué usuarios va dirigido el sistema y tratar de que responda a sus necesidades y preferencias.
- Es recomendable emplear conceptos, términos y simbología que sean familiares para los usuarios.
- Debe mantenerse la coherencia dentro del sistema a desarrollar y, de haberlos, también con los otros sistemas existentes.
- Intentar agrupar las funciones de forma lógica.
- Mostrar información acerca del estado de la ejecución de las funciones.
- Obstaculizar la selección de acciones que puedan ser destructivas de datos o información.
- Generar diálogos eficientes.

El impacto de la confección de prototipos en el ciclo de vida y en los requerimientos

Muchos de los errores detectados en los sistemas provienen de una comunicación deficiente entre el usuario y el analista durante el proceso de obtención de los requisitos. Estos errores se transforman en uno de los tipos más caros de corregir porque se visualizan en forma tardía y pueden multiplicar varias veces lo que hubiera sido el costo inicial de su corrección.

Los problemas más habituales en la comprensión de los requisitos se producen porque se encuentran incompletos en su definición, presentan ambigüedades o son inconsistentes entre ellos y, por lo tanto, incorrectos, ya que no definen las reales necesidades de los usuarios.

La confección de un prototipo puede ser esencialmente útil cuando nos encontramos ante requerimientos que van cambiando rápidamente, en circunstancias donde el cliente se encuentra poco abierto a entregar una lista completa de requerimientos o si sucede que ni el cliente ni nosotros conocemos bien el área de aplicación.

La principal desventaja en el desarrollo de un prototipo es que se torna imposible conocer, desde el comienzo, es cuánto tiempo tomará crear un producto que sea aceptable para el cliente. En este ida y vuelta en la búsqueda de un acuerdo, no sabemos cuántas iteraciones será necesario llevar a cabo.

En un estudio realizado por Gordon y Bieman en el año 1995, y publicado en *Rapid prototyping: lesson learned* (IEEE Software), que realizaron sobre la base del estudio de 39 proyectos de prototipado, determinaron que los beneficios del uso de estos fueron:

- Perfeccionamiento en la usabilidad del nuevo sistema.
- Mayor coherencia entre las necesidades de los usuarios y el sistema.
- Un diseño de mejor calidad.
- Mejorar el proceso de mantenimiento.
- Disminución del esfuerzo requerido en el desarrollo.

La forma de evitar problemas a posteriori se basa en una buena gestión del proceso de requerimientos.

Diseño centrado en el usuario

Al iniciar el proceso de determinación de requerimientos, primero se tiende a revisar los aspectos más visibles del sistema y se trabaja con los usuarios. Por esta razón, es muy importante su colaboración durante el desarrollo. Aquí es donde se determina lo que el cliente verá y las interacciones que tendrá con el sistema; será la herramienta que nos permita obtener un feedback para acordar cuáles son los requerimientos y cómo tratarlos.

Prototipo de pantalla (MOCKUP)

Un mockup es una representación visual, pero estática de un diseño. Son habituales en la realización de diseños web para permitirle a los miembros del equipo de desarrollo tener una idea más clara de cómo se verá el sitio web, mostrando tanto texto como imágenes e íconos.

Este prototipo de pantallas representa, de una forma estática, cómo se estructura la información, cómo se visualizan los contenidos y muestra las funciones básicas que estarán disponibles. Por ejemplo, si quisiera mostrar qué incluirá un elemento como un video, lo que se hará es representarlo con una imagen y un icono de reproducción, por este carácter estático que indicamos que emplea.

De todos modos, debemos destacar que no dejan de ser una herramienta útil para permitirles a los miembros del equipo visualizar el diseño a llevar a cabo, y también para poder emplearlos en la búsqueda de feedback, o comentarios, en las posibles revisiones que se desarrollen con la participación del cliente o usuarios.

Caso de uso

Los casos de uso se emplean para conducir el desarrollo de sistemas de software y han resultado una herramienta extraordinariamente útil en el proceso de definición de requerimientos.

Desde su presentación inicial en el año 1987 por Ivar Jacobson, con su método OOSE (Object Oriented Software Engineering) para el modelado y desarrollo de sistemas orientados a objetos, se fue convirtiendo, con los años, en la base de muchos métodos diferentes y es uno de los diagramas incluidos en UML (Lenguaje Unificado de Modelado). La idea de esta herramienta es el estudio de las interacciones entre el sistema y su entorno. Los casos de uso permiten dejar bien claro qué es lo que hará un sistema y, por omisión intencional, también lo que no hará.

Pero, **¿qué es un caso de uso?** Según la definición brindada por Jacobson, Rumbaugh y Booch, un caso de uso es una descripción de un conjunto de secuencias de acciones incluyendo variantes, que ejecuta un sistema para producir un resultado observable de valor para un actor". (*El lenguaje unificado de modelado*, p. 193)

Así, un caso de uso representa un conjunto de situaciones o escenarios típicos que ayudan a poder estructurar, relacionar y también comprender los requerimientos esenciales del sistema que se modela.

Veamos un ejemplo: si pensamos en una biblioteca que presta libros a sus socios, un caso de uso seguramente será registrar el préstamo del libro. Gráficamente, el caso de uso se representará como una elipse y en su interior estará el nombre asignado al caso, que se expresa desde el punto de vista de quien lo utiliza/realiza. Comienza con verbo en infinitivo, seguido del principal objeto o entidad del sistema, afectado por el caso de uso.

En nuestro ejemplo:

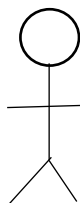


Actores primarios y secundarios

Según los creadores de UML (Jacobson, Rumbaugh y Booch), un actor representa un conjunto coherente de roles que juegan los usuarios de los casos de uso al interactuar con estos. Los actores pueden ser personas o pueden ser sistemas mecánicos”. (*El lenguaje unificado de modelado* p. 192)

Un actor es alguien o algo que interactúa con el sistema. Los actores pueden ser organizaciones, unidades o personas, que es la situación normal cuando se está modelando aplicaciones típicas de sistemas de información para pymes, por ejemplo. Pero también podrían ser otros sistemas o aplicaciones.

Hay que tener presente que si el actor es una persona, nunca colocaremos en nuestro modelo su nombre, sino el rol que representa en su interacción con el sistema, ya que es posible que, en ocasiones, una misma persona pueda interactuar con el sistema desde roles diferentes. Por ejemplo: en su rol, el bibliotecario debe registrar la entrada y salida a su puesto de trabajo, y lo hará desde su rol de “empleado”, pero si, además, en algún momento debe ingresar nuevos libros adquiridos a la biblioteca lo hará desde su rol de “bibliotecario”, que será quien tenga el permiso correspondiente para llevar a cabo esta acción, ya que no cualquier empleado podrá realizarla. Gráficamente, un actor se representa como un muñeco de palo, debajo del cual debe indicarse su rol.



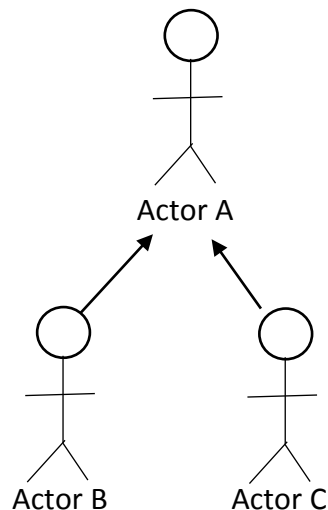
Bibliotecario

Para identificar a los actores, puede ayudar responder a las siguientes preguntas:

- ¿Quién usa el sistema?
- ¿Quién necesita que el sistema le brinde información?
- ¿Quién provee al sistema de información?

- ¿Qué otros sistemas interactúan o utilizan el sistema?
- ¿Quién instala, prepara y mantiene el sistema?

Una cuestión sumamente **importante** a destacar en este momento es que cada caso de uso puede ser iniciado únicamente por UN actor. Si existe más de un actor que pueda iniciar un caso de uso, debe resolverse mediante el empleo de una relación de herencia entre actores, que se graficará de la siguiente manera:



Los actores usan el sistema a través de los casos de uso.

- **Actor primario:** inicia el caso de uso, y utiliza un caso de uso para lograr un objetivo.
- **Actor secundario:** es requerido (usado) por el caso de uso. Se emplea cuando, por ejemplo, como resultado de la ejecución del caso de uso, se requiere notificarlo o enviarle información a otro actor que no fue quien inició el caso.

Descripción del caso de uso (curso normal y alternativas)

Podemos decir que cada caso de uso es, básicamente, una descripción de posibles secuencias de interacciones entre el sistema y sus actores externos, con la finalidad de alcanzar un objetivo determinado.

Hemos visto hasta el momento cómo representar cada caso de uso y, si bien son muy claros en esta vista gráfica, nos falta conocer un poco más en detalle cómo lleva a cabo cada caso de uso esa secuencia de interacciones que indicamos. Es por ello que cada caso de uso cuenta con cuadro descriptivo, que puede ser similar al siguiente:

Nombre del caso de uso: <i>(Debe coincidir con el indicado en el diag. de casos de uso)</i>	
Actor:	
Precondición:	
Curso normal	Curso alternativo
1) ... 2) ... 3) ... 4) ...	3.1.) ...

Primero, identificaremos a qué caso de uso corresponde el cuadro de descripción, indicando su nombre (debe coincidir con el indicado en la elipse). Luego, llegará el momento de indicar quién es el actor que inicio ese caso de uso.

Una vez completado esto, debemos comenzar a describir las interacciones. Siempre comenzaremos por el curso normal, es decir, cómo sería si todas las condiciones o controles salieran bien. En esta descripción debe incluirse tanto lo que hará el actor como lo que hará el sistema. Estos pasos deben numerarse.

Una vez concluida la descripción del curso normal, comenzaremos a ver si en algún paso surgió alguna alternativa que me obligue a cambiar el curso de acción. Será en ese paso donde aparecerá el curso alternativo, que numeraremos en la segunda columna. Por ejemplo: en el cuadro sucedió con el paso 3, por eso la alternativa se indica como 3.1), indicando dónde se produjo ese curso alternativo.

Otro ejemplo de un cuadro para descripción del caso de uso más completo sería el que se muestra a continuación.

Identificador	<identificador del requerimiento>
Nombre	<nombre del requerimiento funcional>
Actor principal	<Aquel que protagoniza el caso de uso>
Actor secundario	<Aquel que siendo actor, no lo inicia>
Descripción	<El sistema deberá comportarse tal como se describe>

Precondición	<precondición del caso de uso>	
Secuencia Normal	Paso	Acción
	1	El <actor> / <sistema> <acción realizada por el actor o el sistema>
	2	
	...	...
Postcondición	<postcondición del caso de uso>	
Excepciones	Paso	Acción
	1	Si <condición de excepción>, el <actor> / <sistema> <acción realizada por el actor o sistema>, se realiza el caso de uso < caso de uso CU-x>, a continuación, este caso de uso {continua, aborta}
	...	...
Importancia	<nivel de importancia>	
Comentarios	<comentarios adicionales>	

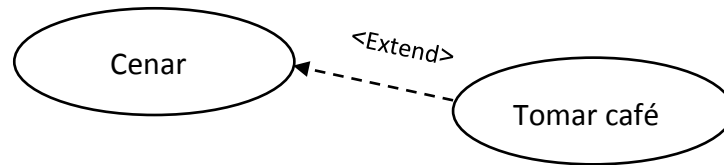
Relaciones de extensión, y uso

Una vez que hayamos identificados los casos de uso principales, comenzaremos a refinar el modelo, tratando de identificar nuevos casos de uso que surgen a partir de los existentes. Las relaciones de extensión y uso se pueden presentar únicamente entre casos de uso.

Un caso de uso extiende a otro cuando tiene una parte de la funcionalidad que no siempre ocurre. Pero aun así, ambos deben ser casos de uso en sí mismos (cumplir con la definición de caso de uso).

El mejor ejemplo para comprender esto es el descrito por Ivar Jacobson, su creador: si tenemos el caso de uso “Cenar”, podría extenderse a un caso de uso “Tomar café”. Si bien podría darse que tomemos una café después de una cena, esto no es un error o excepción sino, simplemente, algo que podría o no darse.

Gráficamente, se representará de la siguiente manera:



Las relaciones de uso se dan cuando un caso de uso está incluido en otro, y se trata en realidad de un escenario común a más de un caso. Al igual que mencionamos previamente, tiene que ser un caso de uso en sí mismo (es decir, que debe cumplir con la definición de caso de uso).

Este tipo de relaciones suelen aparecer después de haber especificado varios casos de uso, y se muestran como una funcionalidad común a más de uno.

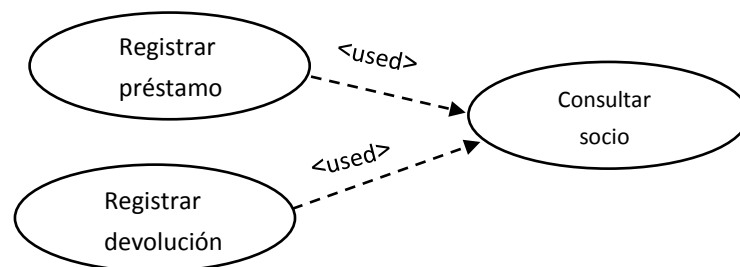


Diagrama de casos de uso

El diagrama de casos de uso se emplea para modelar la vista de los casos de uso del sistema, y muestra: el conjunto de los casos de uso, los actores y las relaciones.

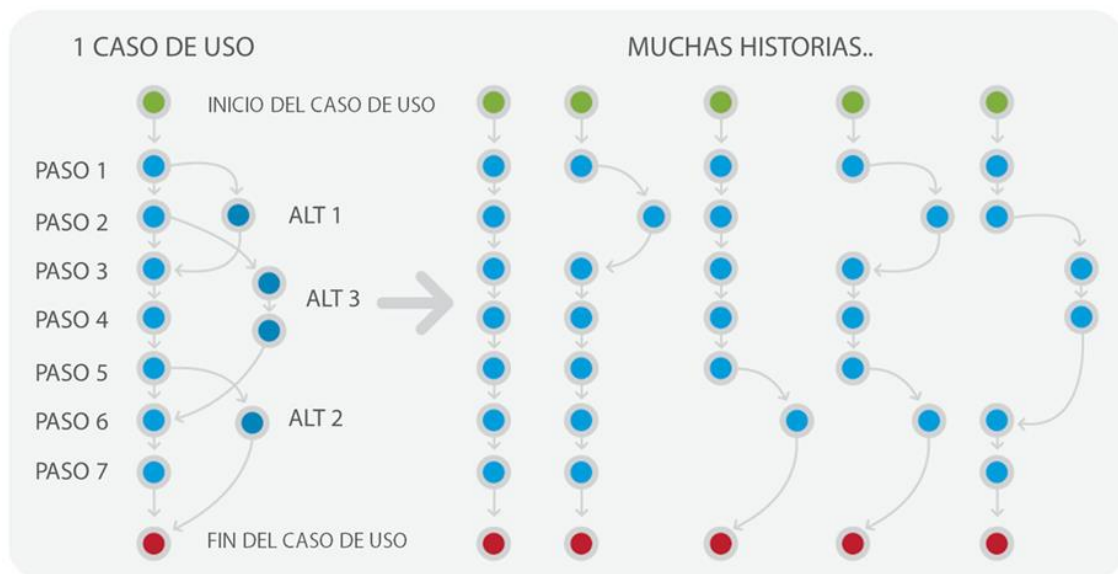
Gráficamente puede emplearse un rectángulo para indicar la frontera entre el sistema y su entorno, e indicar el nombre del sistema en un borde interno de dicho rectángulo. También podrían emplearse cajas rectangulares más pequeñas para agrupar los casos de uso por subsistemas.

Historia de usuario

Para poder comprender un caso de uso, narramos historias. Estas cubren la forma en que se alcanza el objetivo del caso de uso en forma exitosa, así como también como se manejará ante cualquier problema que pudiera ocurrir en el camino.

Los casos de uso 2.0, desarrollados por Ivar Jacobson, fueron diseñados con la intención de aliviar el proceso. Por esa razón, él sugiere escribir narrativas de casos de uso más ligeras que capturen lo que sea estrictamente necesario para poder

generar las historias de usuario, a tal punto que podrían describirse en tarjetas pequeñas y luego bajar detalle en los casos de uso que lo requieran.



Fuente: [https://www.ivarjacobson.com/sites/default/files/field_iji_file/article/use_case_2.0 -
_spanish_translation.pdf](https://www.ivarjacobson.com/sites/default/files/field_iji_file/article/use_case_2.0_-_spanish_translation.pdf)

Las historias de usuario sirven para ayudar a entender el caso de uso y permiten implementarlo porción por porción. Así, en lugar de describir el sistema de una sola vez, podemos ir haciéndolo por cada caso de uso, narrando las historias que cada uno capture. Tomamos la narración de las historias como parte de la narrativa del caso de uso.

Una historia de usuario es la descripción de una funcionalidad desde el punto de vista del usuario o cliente. Se suelen expresar en párrafos breves, idealmente hasta dos oraciones, que se cargan en tarjetas, y deben responder tres preguntas:

- ¿Quién es el beneficiario? Interesados/actores que se beneficiarán de la historia de usuario
- ¿Qué se quiere hacer? Descripción/visión de alto nivel de la funcionalidad para el usuario.
- ¿Cuál es el beneficio? El “por qué” /cual es el valor de negocio que la historia proporciona.

Podrían describirse empleando el siguiente formato, por ejemplo:

Como (rol) yo quiero (algo) para (beneficiarme)

Para el ejemplo de la biblioteca que tratamos antes, podría ser:

Como bibliotecario de la institución yo quiero revisar el historial de préstamos para identificar los libros con mayor demanda para reponer.

Cuando se habla de historias de usuarios surge el concepto de épicas, que se refiere a cuando las historias de usuario son grandes y difícilmente puedan llegar a ser implementadas en una única iteración, es necesario desagregarla en historias de usuario menores.



Fuente: https://www.ivarjacobson.com/sites/default/files/field_iii_file/article/use_case_2.0_-_spanish_translation.pdf

Los atributos de las buenas historias de usuario, según el criterio INVEST creado por Bill Wake en el año 2003, son los siguientes:

- **Independiente:** la historia de usuario debe ser autosuficiente para no presentar dependencia de ninguna otra historia de usuario.
- **Negociable:** las historias de usuario pueden ser cambiadas hasta tanto sean parte de una iteración.
- **Valorable para el usuario o cliente:** la historia de usuario debe entregar valor al cliente o usuario final.
- **Estimable:** siempre debería poder ser capaz de estimar cuál es el esfuerzo de la historia de usuario.
- **Pequeña:** las historias de usuario nunca deben ser tan grandes como para ser imposibles de planificarse o priorizarse.
- **Comprobable:** la historia de usuario, o su descripción, debe brindar la información necesaria para poder probar su desarrollo efectivo.